

SIMULATING LOW PRECISION FLOATING-POINT ARITHMETIC*

NICHOLAS J. HIGHAM[†] AND SRIKARA PRANESH[†]

Abstract. The half-precision (fp16) floating-point format, defined in the 2008 revision of the IEEE standard for floating-point arithmetic, and a more recently proposed half-precision format bfloat16, are increasingly available in GPUs and other accelerators. While the support for low precision arithmetic is mainly motivated by machine learning applications, general purpose numerical algorithms can benefit from it, too, gaining in speed, energy usage, and reduced communication costs. Since the appropriate hardware is not always available, and one may wish to experiment with new arithmetics not yet implemented in hardware, software simulations of low precision arithmetic are needed. We discuss how to simulate low precision arithmetic using arithmetic of higher precision. We examine the correctness of such simulations and explain via rounding error analysis why a natural method of simulation can provide results that are more accurate than actual computations at low precision. We provide a MATLAB function, `chop`, that can be used to efficiently simulate fp16, bfloat16, and other low precision arithmetics, with or without the representation of subnormal numbers and with the options of round to nearest, directed rounding, stochastic rounding, and random bit flips in the significand. We demonstrate the advantages of this approach over defining a new MATLAB class and overloading operators.

Key words. floating-point arithmetic, half precision, low precision, IEEE arithmetic, fp16, bfloat16, subnormal numbers, mixed precision, simulation, rounding error analysis, round to nearest, directed rounding, stochastic rounding, bit flips, MATLAB

AMS subject classifications. 65F05, 65G50, 65Y04

DOI. 10.1137/19M1251308

1. Introduction. The 1985 IEEE standard 754 for floating-point arithmetic [28] brought to an end the turbulent period in scientific computing when hardware manufacturers provided different floating-point arithmetics, some of which were better behaved than others. A particular problem had been that the lack of a guard digit in some floating-point arithmetics could cause algorithms that otherwise work perfectly to fail [24, sect. 2.4]. For the two decades following the introduction of the standard, virtually all floating-point hardware implemented IEEE single precision (fp32) or double precision (fp64) arithmetic.

The 2008 revision of the standard [29] defined a 16-bit half-precision (fp16) storage format but did not define operations on it. However, manufacturers have provided implementations of half-precision arithmetic, using the natural extension of the rules for single and double precision. Half precision is available, for example, on the NVIDIA P100 (2016) and V100 (2017) GPUs, the AMD Radeon Instinct MI25 GPU (2017), and the ARM NEON architecture [1].

With regard to future chips, the Fujitsu A64FX Arm processor that will power Japan's first exascale computer will employ half precision [10]. Also, the next

*Submitted to the journal's Software and High-Performance Computing section March 20, 2019; accepted for publication (in revised form) July 30, 2019; published electronically October 29, 2019. The opinions and views expressed in this publication are those of the authors, and not necessarily those of the funding bodies.

<https://doi.org/10.1137/19M1251308>

Funding: This work was supported by MathWorks, by Engineering and Physical Sciences Research Council grant EP/P020720/1, and by the Royal Society.

[†]School of Mathematics, University of Manchester, Manchester, M13 9PL, UK (nick.higham@manchester.ac.uk, srikara.pranesh@manchester.ac.uk).

generation AI chips of IBM will support an 8-bit floating-point format along with half precision [11], [58].

Another form of half-precision arithmetic, called bfloat16,¹ was introduced by Google in its tensor processing units and will be supported by Intel in its forthcoming Nervana Neural Network Processor [45] and Cooper Lake processors [12]. The same floating-point format is suggested for ultra low power computing in [54], where it is called binary16alt.

While machine learning provides much of the impetus for the development of half-precision arithmetic in hardware [16], [52], half precision is also attractive for accelerating general purpose scientific computing. It is already being used in weather forecasting and climate modeling [4], [7], [20], [44], [55] and the solution of linear systems of equations [2], [3], [17], [18], [19], [25]. The Summit machine at Oak Ridge National Laboratory, which leads the June 2018, November 2018, and June 2019 Top 500 lists (<https://www.top500.org>), has a peak performance of 148.6 petaflops in the LINPACK benchmark,² a benchmark that employs only double precision. For a genetics application that uses half precision, the same machine has a peak performance of 2.36 exaflops [13]. This difference in the peak performances highlights the importance of using low precision arithmetic to exploit the hardware. Library software that makes efficient use of these low precision formats is being developed—for example, MAGMA³ [8], [56], [57].

While low precision enabled hardware can provide substantial computational benefits, these are specialist devices to which not everyone has ready access. Furthermore, one may wish to experiment with new floating-point formats not yet available in hardware. In this work we investigate and propose ways to simulate low precision floating-point arithmetics. Our approach is general but with a particular focus on numerical linear algebra kernels and implementation in MATLAB.

We propose in section 3 two simulation strategies, in both of which computations are carried out at higher precision and then rounded to lower precision. In the first variant, Simulation 3.1, we perform every scalar operation—addition, subtraction, multiplication, and division—in high precision and round the results to low precision. We explain why double rounding could vitiate this approach and identify precisions for which it provides a valid simulation. In the second variant, Simulation 3.2, we perform certain vector, matrix, or tensor kernels entirely in high precision and then round the final answer to low precision. Using rounding error analysis we analyze the accuracy and backward stability of matrix-matrix multiplication and solution of a triangular system when the second form of simulation is used. We show that the simulation provides results with forward errors that are essentially independent of n and for triangular systems are also independent of the condition number of the triangular matrix, so the results will generally be more accurate than for true low precision computations.

In section 4 we discuss the pros and cons of Moler's fp16 and vfp16 classes (data types) for MATLAB, which use a combination of Simulations 3.1 and 3.2. In section 5 we propose an alternative approach to simulating low precision arithmetic that stores data in single or double precision, computes at that precision, and rounds the results to match the target precision and range. We provide a MATLAB function, `chop`, that rounds its argument to fp16, bfloat16, fp32, fp64, or a user-defined format with round

¹https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

²<https://www.top500.org/lists/2019/06/>

³<https://icl.utk.edu/magma/software/index.html>

TABLE 2.1

Parameters for bfloat16, fp16, fp32, and fp64 arithmetic: number of bits in significand and exponent, unit roundoff u , smallest positive (subnormal) number $x_{\min}^s$, smallest normalized positive number $x_{\min}$, and largest finite number $x_{\max}$, all given to three significant figures.

	Signif.	Exp.	u	$x_{\min}^s$	$x_{\min}$	$x_{\max}$
bfloat16	8	8	3.91×10^{-3}	$9.18 \times 10^{-41} \dagger\dagger$	1.18×10^{-38}	3.39×10^{38}
fp16	11	5	4.88×10^{-4}	5.96×10^{-8}	6.10×10^{-5}	6.55×10^4
fp32	24	8	5.96×10^{-8}	1.40×10^{-45}	1.18×10^{-38}	3.40×10^{38}
fp64	53	11	1.11×10^{-16}	4.94×10^{-324}	2.22×10^{-308}	1.80×10^{308}

$\dagger\dagger$ In Intel's bfloat16 specification subnormal numbers are not supported [30], so any number less than $x_{\min}$ in magnitude is flushed to zero. The value shown would pertain if subnormals were supported.

to nearest, round towards plus or minus infinity, round towards zero, or stochastic rounding, and with optional support of subnormal numbers. The function also allows random bit flips in the significand in order to simulate soft errors, which could be due to running a processor at a reduced voltage [43]; such bit flips are explored in [55], for example. Computing reliably in the presence of soft errors is of interest in training large-scale neural networks [9]. In section 6 we demonstrate the performance advantages of `chop` over `fp16/vfp16` and give some examples of the insight that can be gained by experimenting with low precision computations in MATLAB.

2. Low precision floating-point arithmetics. We assume that IEEE standard single precision and double precision are available in hardware and that we wish to simulate arithmetics of lower precision. Table 2.1 shows the arithmetics of particular interest in this work. Even lower precisions, including 8-bit (quarter precision) formats [36], [38], [54], [58] and a 9-bit format [27], are also of interest, but they are not standardized.

We wish to simulate arithmetic of the appropriate precision and range, with various rounding modes, and make the support of subnormal numbers optional. IEEE arithmetic supports subnormal numbers, but bfloat16 as defined by Intel does not [30]. The main reason for a hardware designer not to support subnormal numbers is to simplify the implementation, but supporting subnormal numbers in software can degrade performance.⁴ One question that simulation can help address is to what extent subnormal numbers make a difference to the behavior of an algorithm for particular types of data.

Software for arbitrary precision floating-point arithmetic can in principle be used at low precisions. For MATLAB, the VPA arithmetic in the Symbolic Math Toolbox [53] and the Multiprecision Computing Toolbox [42] support arbitrary precisions, but the precision is specified via the number of base 10 digits, and the exponent limits cannot be controlled, so it is not possible to use them to simulate a particular binary low precision format. GNU MPFR [15] is a C library that was originally intended for high precisions, though recent work has improved its efficiency for fp64 and fp128 [34]. As the documentation notes,⁵ the default exponent range is wide and “subnormal numbers are not implemented (but can be emulated).”

⁴<https://devblogs.nvidia.com/cuda-pro-tip-flush-denormals-confidence/>, https://en.wikipedia.org/wiki/Denormal_number

⁵<https://www.mpfr.org/mpfr-current/mpfr.html>

3. Simulation models and their properties. We discuss two different ways to simulate low precision arithmetic. We need to achieve both the correct precision and the correct range. Correct range refers not only to ensuring that numbers too large for the target range overflow but also that numbers in the subnormal range are correctly represented with less precision than normalized numbers. Our approach is to carry out all computations in single precision or double precision, which we refer to as higher precision, and round back to the target precision. This can be done at two different levels of granularity: by rounding every individual scalar operation or by allowing a group of operations to be carried out in higher precision and then rounding the result to lower precision. We investigate how these two possibilities affect the accuracy and stability of computations.

3.1. Rounding every operation. The most natural way to simulate low precision arithmetic is as follows. Here, rounding means any of the following four IEEE arithmetic rounding modes: round to nearest with ties broken by rounding to an even least significant bit, round towards plus infinity or minus infinity, and round towards zero. We will refer to the low precision format of interest as the target format and we assume that the operands are given in the target format. In practice, numbers in the target format might actually be stored in a higher precision format, padded with zero bits (we will use this approach in section 5).

SIMULATION 3.1. Simulate every scalar operation by converting the operands to fp32 or fp64, carrying out the operation in fp32 or fp64, then rounding the result back to the target format.

By computing a result first in fp32 or fp64 and then rounding to the target format we are rounding the result twice. It is well known that double rounding can give a result different from rounding directly to the target format. A simple example with round to nearest in base 10 is rounding 1.34905 to two significant figures: rounding first to three significant figures gives 1.35 and then rounding to two significant figures gives 1.4, but rounding directly to two significant figures gives 1.3. Fortunately, for the operations of addition, subtraction, multiplication, division, and square root rounding first to fp32 or fp64 and then to bfloat16 or fp16 gives the same result as rounding directly to bfloat16 or fp16, even allowing for subnormal numbers. This is shown by results in [14], [47], [50], which essentially require that the format used for the first rounding has a little more than twice as many digits in the significand as the target format (this condition is needed for round to nearest but not for directed rounding). Since double rounding does not change results in simulating bfloat16 or fp16 via fp32 or fp64, we can conclude that Simulation 3.1 is equivalent to carrying out the computation wholly in bfloat16 or fp16, provided that the range is correctly handled.

If the low precision numbers are stored in a special format then Simulation 3.1 can be expensive because every scalar operation incurs the overhead of converting between different floating-point formats.

A subtlety arises in elementary function evaluations, in that Simulation 3.1 is likely to provide a correctly rounded result, whereas a computation entirely at the target precision may be unable to do so. For example, in an exponentiation x^y it is hard to guarantee a correctly rounded result without using extra precision [41, sect. 13.2]. However, since elementary functions are not included in the IEEE standard, the computed results will in any case depend upon the implementation.

Simulation 3.1 is used for Julia's built-in float16 (fp16) data type [32], as well as for the Julia bfloat16 package [33]. A C++ header file for fp16 is available [46];

we note that it does not convert to fp16 after every scalar operation but retains a number in fp32 as long as possible, in order to reduce the overhead caused by repeated conversions. This implementation therefore is not of the form Simulation 3.1. SciPy has a float16 data type,⁶ which appears to use Simulation 3.1 with computations in single precision.

Another instance of Simulation 3.1 is the rpe (reduced floating-point precision) library of Dawson and Düben [6], which provides a derived type and overloaded operators for Fortran and was developed for use in weather and climate modeling. It emulates the specified precision but in general uses the exponent range of double precision.

3.2. Rounding kernels. We now consider a weaker form of simulation. Here, “kernel” can be a vector, matrix, or tensor operation.

SIMULATION 3.2. *Simulate isolated scalar operations as in Simulation 3.1. Implement appropriate kernels by carrying out the kernel in fp32 or fp64 and then rounding the result back to the target precision.*

The motivation for Simulation 3.2 is efficiency and simplicity, since the chosen kernels can be executed in whatever way the underlying environment provides. With this form of simulation we could compute the product of two matrices A and B by converting A and B to fp64, computing the product in fp64, and then rounding the result back to the target format.

We now consider what can be said about the accuracy and backward stability of two particular kernels under Simulation 3.2: matrix multiplication and the solution of a triangular system.

Denote the lower precision by fp_{low} and the higher precision (fp32 or fp64) by fp_{high}. We assume round to nearest, use the standard model of floating-point arithmetic [24, sect. 2.2], and ignore the possibility of overflow and underflow. We denote by u_ℓ and u_h the unit roundoffs for the lower and higher precisions and write

$$\gamma_\ell(n) = \frac{nu_\ell}{1 - nu_\ell}, \quad \gamma_h(n) = \frac{nu_h}{1 - nu_h}$$

for $nu_\ell < 1$ and $nu_h < 1$.

Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, and let $C = AB$. The product $\hat{C}$ computed in precision fp_{high} satisfies [24, sect. 3.5]

$$(3.1) \quad |C - \hat{C}| \leq \gamma_h(n)|A||B|,$$

where the absolute value is taken componentwise. Rounding $\hat{C}$ to $\tilde{C}$ in fp_{low}, we have [24, Thm. 2.2]

$$(3.2) \quad \tilde{C} = \hat{C} + \Delta C, \quad |\Delta C| \leq u_\ell |\hat{C}|.$$

Hence

$$|C - \tilde{C}| = |C - \hat{C} + \hat{C} - \tilde{C}| \leq \gamma_h(n)|A||B| + u_\ell |\hat{C}|.$$

From (3.1) we obtain

$$|\hat{C}| \leq (1 + \gamma_h(n))|A||B|$$

and then

$$|C - \tilde{C}| \leq (\gamma_h(n) + u_\ell(1 + \gamma_h(n)))|A||B|.$$

⁶<https://docs.scipy.org/doc/numpy-1.15.4/user/basics.types.html>

Hence we certainly have

$$(3.3) \quad |C - \tilde{C}| \leq 2u_\ell |A| |B|$$

as long as $\gamma_h(n)(1 + u_\ell) + u_\ell \leq 2u_\ell$, which is equivalent to

$$(3.4) \quad n \leq \frac{u_\ell}{u_h(1 + 2u_\ell)}.$$

When `fp_low` and `fp_high` are `fp16` and `fp64`, respectively, this bound is 4.3×10^{12} , which covers all n of current practical interest for dense matrix multiplication. The bound (3.3) is to be compared with the bound

$$(3.5) \quad |C - \hat{C}| \leq \gamma_\ell(n) |A| |B|$$

that holds for the computation done entirely in `fp_low`. The difference is that in (3.3) the constant does not depend on n , as long as (3.4) holds. Looked at another way, the constant in (3.5) exceeds 1 once $nu_\ell \geq 1$ (indeed, the bound is only valid for $nu_\ell < 1$), whereas the constant in (3.3) is of order u_ℓ and the bound holds for n up to a much larger value. This demonstrates that the results from Simulation 3.2 will in general be more accurate than for actual low precision computation.

Now consider the solution of a triangular system $Tx = b$, where $T \in \mathbb{R}^{n \times n}$ and $x, b \in \mathbb{R}^n$. Solving by substitution in `fp_high` we know that the computed solution $\hat{x}$ satisfies the componentwise backward error result [24, Thm. 8.5]

$$(3.6) \quad (T + \Delta T)\hat{x} = b, \quad |\Delta T| \leq \gamma_h(n)|T|.$$

Rounding to `fp_low` gives

$$(3.7) \quad \tilde{x} = (I + D)\hat{x}, \quad D = \text{diag}(\delta_i), \quad |\delta_i| \leq u_\ell.$$

First, we consider the backward error. We have

$$|T\tilde{x} - b| = |T\hat{x} + TD\hat{x} - b| = |-\Delta T\hat{x} + TD\hat{x}| \leq (\gamma_h(n) + u_\ell)|T||\hat{x}|,$$

which implies that [24, Thm. 7.3]

$$(3.8) \quad (T + \Delta T_h)\tilde{x} = b, \quad |\Delta T_h| \leq (u_\ell + \gamma_h(n))|T|.$$

If (3.4) holds then we certainly have $|\Delta T_h| \leq 2u_\ell|T|$.

The conclusion is that $\tilde{x}$ has the same componentwise form of backward error as if it had been computed entirely in `fp_low`, but with no dependence on n as long as (3.4) holds.

Turning to the forward error, (3.6) implies [24, Thm. 7.4]

$$(3.9) \quad \frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \frac{\text{cond}(T, x)\gamma_h(n)}{1 - \text{cond}(T)\gamma_h(n)},$$

where

$$\text{cond}(T, x) = \frac{\| |T^{-1}| |T| |x| \|_\infty}{\|x\|_\infty}, \quad \text{cond}(T) = \| |T^{-1}| |T| \|_\infty.$$

Combined with (3.7) this gives

$$\frac{\|x - \tilde{x}\|_\infty}{\|x\|_\infty} \lesssim \frac{\text{cond}(T, x)\gamma_h(n)}{1 - \text{cond}(T)\gamma_h(n)} + u_\ell.$$

Hence, since $\text{cond}(T, x) \leq \text{cond}(T)$, as long as $\gamma_h(n) \text{cond}(T) \lesssim u_\ell$, that is,

$$(3.10) \quad n \text{cond}(T) \lesssim \frac{u_\ell}{u_h},$$

we will have

$$(3.11) \quad \frac{\|x - \tilde{x}\|_\infty}{\|x\|_\infty} \lesssim 2u_\ell,$$

and the right-hand side of (3.10) is 4.3×10^{12} for `fp_low` as `fp16` and `fp_high` as `fp64`. This is to be compared with the bound

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \frac{\text{cond}(T, x) \gamma_\ell(n)}{1 - \text{cond}(T) \gamma_\ell(n)}$$

(analogous to (3.9)) that we would expect for a solution computed entirely in `fp_low`. Hence when (3.10) is satisfied we can expect $\tilde{x}$ to be more accurate than it would be if it were computed entirely in `fp_low`, for two reasons: the constant term in (3.11) does not depend on n and the condition number does not appear in the bound.

Suppose now that T is upper triangular and that we implement back substitution for $Tx = b$ as

$$x_i = \text{fl}_{\text{low}} \left(\frac{b_i - \text{fl}_{\text{low}} \left(\text{fl}_{\text{high}} \left(\sum_{j=i+1}^n t_{ij} x_j \right) \right)}{t_{ii}} \right);$$

that is, the underlying inner product is evaluated in `fp_high` but the subtraction and division are in `fp_low`. It is easy to see that we now obtain a backward error result of the same form as (3.8) but with a different constant that again is independent of n unless n is very large, but we cannot conclude that a small normwise forward error is obtained. So this mode of evaluation is closer to evaluation wholly in `fp_low` than if we execute the substitution in `fp_high` and then round to `fp_low`.

In summary, our analysis of Simulation 3.2 shows that the results it yields can be more accurate than those from true low precision computation in two respects. First, by evaluating the whole kernel at precision `fp_high` we obtain a forward error bound for matrix multiplication and a backward error bound for the solution of triangular systems that have essentially no dependence on n . Second, the forward error for $Tx = b$ will be essentially independent of the condition number. The second phenomenon no longer holds if only the underlying inner products are evaluated at the higher precision.

Our conclusion is that care is needed in deciding at what level of granularity to apply the higher precision. The lower the level, the more realistic the simulation.

4. Moler's `fp16` and `vfp16` classes. We now focus on simulating low precision arithmetic in MATLAB. The `fp16` half-precision MATLAB class of Moler [36], [38] introduces a new data type `fp16` that implements the IEEE `fp16` storage format and overloads some basic functions for `fp16` arguments. It uses a mixture of Simulations 3.1 and 3.2. Although it was designed to be economical in coding rather than to have good performance, the `fp16` class has proved very useful in research [3], [25], [26].

The `fp16` class includes the following two functions (which live in the `@fp16` directory).

```
function z = plus(x,y)
    z = fp16(double(x) + double(y));
end

function [L,U,p] = lu(A)
    [L,U,p] = lutx(A);
end
```

These functions do the computations in double precision and round the result back to `fp16`. The `plus` function is used for matrix addition and is invoked when MATLAB encounters the `+` operator with `fp16` operands. The `lutx` function called from `lu` is a “textbook” implementation of LU factorization with partial pivoting, implemented as a straightforward MATLAB program file in [36]. When this function is called with an `fp16` argument, all the operations within the function are done by the overloaded `fp16` operators, and `fp16` outputs are returned. The key lines of `lutx` are

```
% Compute multipliers
i = k+1:n;
A(i,k) = A(i,k)/A(k,k);

% Update the remainder of the matrix
j = k+1:n;
A(i,j) = A(i,j) - A(i,k)*A(k,j);
```

For `fp16` variables, the multiplication in the last of these lines calls the function `mtimes` in the `@fp16` directory:

```
function z = mtimes(x,y)
    z = fp16(double(x) * double(y));
end
```

Because `mtimes` is being called to evaluate a rank-1 matrix, and hence it performs one multiplication per entry and no additions, that line faithfully simulates `fp16` arithmetic. However, an expression `A*B` with `fp16` matrices `A` and `B` will in general be evaluated as multiple double precision multiplications and additions with a final rounding to `fp16`, that is, as in Simulation 3.2 with matrix multiplication as the kernel.

The solution of linear systems (as invoked by backslash) is done by the function `mldivide`, which has the form

```
function z = mldivide(x,y)
    z = fp16(double(x) \ double(y));
end
```

Here again, the whole solution process is carried out in double precision and then rounded to `fp16`, so this is Simulation 3.2 with linear system solution as the kernel. This is not equivalent to carrying out the computation in `fp16`, as we saw in the previous section in the special case of triangular systems.

From our analysis in the previous section we conclude that in the `fp16` class both matrix multiplication and linear system solutions enjoy error bounds that are

TABLE 5.1

Rounding modes supported by MATLAB function `chop`, as specified by the function's `options.round` parameter.

<code>options.round</code>	Rounding modes
1	Round to nearest, with tie broken by rounding to the number with an even last bit
2	Round towards ∞
3	Round towards $-\infty$
4	Round towards zero
5	Stochastic rounding, mode 1: round to the next larger or smaller floating-point number x with probability equal to 1 minus the distance to x
6	Stochastic rounding, mode 2: round up or down with equal probability.

essentially independent of n , and the forward error for the latter will be condition number independent (for all n of practical interest), so we can expect to obtain more accurate results than for actual fp16 computation.

Moler's fp16 class has the advantage that it is a true MATLAB class, and so program files (ones containing sequences of MATLAB commands) can work seamlessly when provided with fp16 input arguments. However, arithmetic in this class is very slow because of both the overhead of object orientation in MATLAB and the cost of converting to and from the fp16 storage format. In the experiments with linear systems in [25], [26] the dimension was restricted to a few hundred in order to obtain reasonable computing times.

Moler has also written a class `vfp16` that allow the partitioning of a 16-bit word between significand and exponent to be varied, in particular allowing bfloat16 to be simulated [40]. This class also allows subnormals to be supported or not, and it allows fused multiply-adds (FMAs) to be done within the inner products inside a matrix multiplication.

5. Parametrized rounding. We now develop a different strategy to simulate low precision arithmetic in MATLAB. Instead of defining a new data type, we store all variables in double precision or single precision but ensure that they have the precision and range corresponding to the lower precision. Computations are performed in the storage format but rounded to the target precision.

This strategy is facilitated by our MATLAB function `chop`. It takes an fp64 or fp32 argument and rounds it to a precision chosen from among bfloat16, fp16, fp32, fp64, or a custom format, using one of the rounding modes listed in Table 5.1. In the custom defined format, the number of bits in the significand and the maximum value of the exponent are specified by the user. This custom format allows selection of, for example, the nonstandard 8-bit and 16-bit formats proposed in [58]. To avoid double rounding, the number of bits in the mantissa of the custom format must be less than half that of the format in which the data is given, which is either single or double. We support stochastic rounding because of its increasing adoption in machine learning [16] and the need for better understanding of its behavior. See the appendix for a link to `chop` and examples of its usage.

Note that MATLAB has an undocumented function `feature('setround',mode)`, (as used in [49], for example), which sets the rounding mode on the processor. Here, `mode` can specify any of the first four rounding modes in Table 5.1. This function

works for the built-in `fp32` and `fp64` data types only, so it is not useful within `chop`, but we have used it to test `chop`.

The `chop` function allows subnormal numbers to be supported or flushed to zero. The reason we allow `fp64` as an output option is that by asking for subnormals not to be supported one can explore the effect of flushing subnormals to zero in `fp64`. The `chop` function can also create random bit flips in the significand, which enable soft errors to be simulated.

Chopping refers to rounding towards zero, which was the form of rounding used in certain computer architectures prior to the introduction of the IEEE standard. The less specific name `round` would therefore be preferable to `chop`, but `round` is a built-in MATLAB function that rounds to the nearest integer or to a specified number of decimal places. The `chop` function is consistent with earlier MATLAB functions of the same name. The original Fortran version of MATLAB had a function `chop` [37], [39]:

The result of each floating point operation may also be “chopped” to simulate a computer with a shorter word length . . . the statement `CHOP(p)` causes the `p` least significant octal or hexadecimal digits in the result of each floating point operation to be set to zero.

This function was implemented by using the Fortran binary operator `.AND.` to mask the result of every arithmetic operation. Our function `chop` builds on a function of the same name in the Matrix Computation Toolbox [21] that has been successfully used to simulate single precision arithmetic⁷ with `round` to nearest in, for example, [5], [22], [23], [24].

Applying `chop` to the result of every arithmetic operation can be cumbersome. For example, the scalar computation `x = a + b*c` would be rewritten

```
x = chop(a + chop(b*c))
```

However, if an algorithm is expressed in a vectorized fashion, just a few invocations of `chop` may be required. Moler’s function `lutz`, discussed in the previous section, is a good example. To produce a modified function `lutz_chop` that uses `chop` we need to change just two lines, as shown in this extract:

```
% Compute multipliers
i = k+1:n;
A(i,k) = chop(A(i,k)/A(k,k));

% Update the remainder of the matrix
j = k+1:n;
A(i,j) = chop(A(i,j) - chop(A(i,k)*A(k,j)));
```

The expression `chop(A(i,k)*A(k,j))` forms a rank-1 matrix in double precision and then rounds every element to the target precision.

For optimal efficiency we need to minimize the number of calls to `chop`, and this may require reformulating a computation. For example, consider the multiplication $C = AB$ of $n \times n$ matrices. If we code at the scalar level, $C(i,j) = \sum_{k=1}^n A(i,k)B(k,j)$, then $O(n^3)$ calls to `chop` will be required, as `chop` will be applied to the result of every scalar multiplication and addition. Using the formulation

⁷MATLAB first provided single precision arithmetic in MATLAB 7.0 (Release 14) in 2004.

$C(:, j) = \sum_{k=1}^n A(:, k)B(k, j)$ requires $O(n^2)$ calls, but the outer product formulation $C = \sum_{k=1}^n A(:, k)B(k, :)$ requires just $O(n)$ calls to `chop` (analogously to `lutx_chop`) and so is the most efficient way to code matrix multiplication with `chop`.

We discuss the key features of the function `chop`. The general form of a call is `chop(x, options)`, where `options` is a structure used to define the arithmetic format, the rounding mode, whether subnormal numbers are supported, and whether random bit flips are required. The second input argument, `options`, can be omitted as long as the required options are set up on the initial call to `chop`. A persistent variable in the function stores the selected options for future use during a session. Importantly, all arithmetic operations in `chop` are vectorized, so best efficiency is obtained by calling it with a vector or matrix argument. The rounding operation is done in the separate function `roundit`.

A significant advantage of `chop` over the use of a class is that since all data is maintained in single precision or double precision, the overheads of conversions to and from other formats are avoided.

The simple nature of `chop` provides some benefits. Mixed precision arithmetic is easily implemented, simply by using different `options` arguments on different invocations of `chop`. An FMA $y = a + b*c$ can be simulated using `chop(a + b*c)`: assuming the target precision is single or less and that `a`, `b`, and `c` are outputs from `chop` at that precision, the product `b*c` will be computed exactly at double precision and so `a + b*c` will be obtained correctly rounded to double precision and will then be correctly rounded to the target precision.

We note that Rump includes similar functionality to `chop` in his MATLAB toolbox INTLAB for interval arithmetic [48], [50]. The function `flround` therein rounds a double precision variable to a specified precision and range. It does not support stochastic rounding, random bit flips, or turning off of subnormal numbers.

6. Experiments. We now describe experiments with the `chop` function that illustrate some of the points made in the previous sections and show how insight into low precision computations can be gained.

Our experiments were carried out in MATLAB R2018b on a machine with an Intel Core i7-6800K CPU with six cores running at 3.4-3.6GHz. Our codes are available at https://github.com/SrikaraPranesh/LowPrecision_Simulation.

We note that the defaults for `chop` are to support subnormals for `fp16`, `fp32`, and `fp64` and not to support them for `bfloat16`.

The first experiment compares two implementations of matrix multiplication in `fp16` arithmetic: the first carries out every operation in `fp16` and the second computes the product in `fp64` and then rounds to `fp16`. The error bound in (3.3) for the latter computation is constant, whereas for `fp16` the error bound in (3.5) is proportional to n . The results in Table 6.1 for random `fp16` A and B confirm the behavior suggested by the bounds, with the `fp64` $\rightarrow$ `fp16` errors approximately constant and the `fp16` errors growing proportional to n .

Table 6.2 gives the times to LU factorize with partial pivoting an $n \times n$ matrix with random `fp16` entries from the normal $(0, 1)$ distribution using Moler's `lutx` function called by his `fp16` class and using `lutx_chop` with `fp16` arithmetic and with subnormal numbers supported. Results generated by the `lutx` function and our function `lutx_chop` are bitwise identical. We see that `lutx_chop` is of order 1000 times faster than `lutx`. Clearly, the code using the `fp16` class is spending almost all its time on overheads and not on floating-point arithmetic. The last column of the table shows that the conversions to `fp16` in `chop` cause a factor 10 or so slowdown compared with

TABLE 6.1

The error measure $\min\{\epsilon > 0 : |C - \hat{C}| \leq \epsilon|A||B|\}$ for random $n \times n$ A and B from the uniform $[0, 1]$ distribution, where $C = AB$ and $\hat{C}$ is the computed product. The product C is computed entirely in fp16 or in fp64 and then rounded to fp16 (“ $\text{fp64} \rightarrow \text{fp16}$ ”).

n	fp16	$\text{fp64} \rightarrow \text{fp16}$
500	1.19e-02	4.88e-04
1000	2.30e-02	4.88e-04
1500	4.07e-02	3.67e-04
2000	4.60e-02	4.88e-04
2500	6.88e-02	4.29e-04

TABLE 6.2

Time in seconds for LU factorizing with partial pivoting an $n \times n$ matrix `randn(n)` using Moler’s `lutx` function. The function `lutx_chop` is `lutx` modified with calls to `chop`, as discussed in section 5, with fp16 arithmetic and subnormals supported.

n	<code>lutx</code> with fp16 class	<code>lutx_chop</code> at fp16	<code>lutx</code> in double
50	6.3	3.8e-2	1.6e-3
100	4.9e1	4.9e-2	3.2e-3
250	7.2e2	3.4e-1	3.7e-2
500	5.8e3	3.4	4.1e-1

TABLE 6.3

Computed harmonic series $\sum_{i=1}^{\infty} 1/i$ in five precisions of floating-point arithmetic and number of terms for the computed sum to converge.

Arithmetic	Sum	Terms
fp8	3.5000	16
bfloat16	5.0625	65
fp16	7.0859	513
fp32	15.404	2097152
fp64	34.122	$2.81 \cdots \times 10^{14}$

running the code in double precision. This is comparable to the typical slowdown reported for the `rpe` library [6], which is compiled Fortran rather than semicompiled MATLAB.

The ability to vary the precision with which a code executes can be useful in a variety of situations. As a simple example, consider the harmonic series $1 + 1/2 + 1/3 + \cdots$. The series diverges, but when summed in the natural order in floating-point arithmetic it converges, because the partial sums grow while the addends decrease, and eventually the addends are small enough that they do not change the partial sum. Table 6.3 shows the computed harmonic series for five different precisions, along with how many terms are added before the sum becomes constant. The results for the first four arithmetics were computed with MATLAB using the `chop` function (and the `bfloat16` and `fp16` results have been reproduced in Julia [51]). The arithmetic denoted by fp8 is not standard but is a form of quarter precision implemented in Moler’s fp8 class [36], [38]; it apportions five bits to the significand and three to the exponent. The fp64 value is reported by Malone [35], based on a computation that took 24 days.

In Table 6.4 we focus on `bfloat16` and `fp16` and show the computed harmonic series for all the different rounding modes supported by `chop`. We see significant variation of the result with the rounding mode. We declare convergence when two successive

TABLE 6.4

Computed harmonic series $\sum_{i=1}^{\infty} 1/i$ in fp16 and bfloat16 arithmetic for six rounding modes, with number of terms for the computed sum to converge. Round towards zero is equivalent to round towards $-\infty$ in this example. The stochastic rounding modes are defined in Table 5.1.

Rounding mode	bfloat16		fp16	
	Sum	Terms	Sum	Terms
To nearest	5.0625	65	7.0859	513
Towards ∞	2.2×10^{12}	5013	inf	13912
Towards $-\infty$	4.0000	41	5.7461	257
Stochastic, mode 1	4.1875	38	6.1797	277
Stochastic, mode 2	4.0938	34	6.2148	257

TABLE 6.5

The computed sum $\sum_{i=1}^n 1/(n-i+1)^2$ in fp16. $\hat{s}_1$: with subnormal numbers. $\hat{s}_2$: without subnormal numbers.

n	$\hat{s}_1$	$\hat{s}_2$	$\hat{s}_1 - \hat{s}_2$
100	1.635742e+00	1.635742e+00	0.00e+00
1000	1.644531e+00	1.637695e+00	6.84e-03
10000	1.645508e+00	1.637695e+00	7.81e-03

computed partial sums are equal. With stochastic rounding the computed sum might nevertheless increase if more terms were computed.

Next, we show the value of being able to control whether or not subnormal numbers are supported. We evaluate the sum $\sum_{i=1}^n 1/(n-i+1)^2$ for $n = 10^2, 10^3, 10^4$ in fp16 arithmetic, with and without subnormal numbers, using the `chop` function. As Table 6.5 shows, for $n = 10^3, 10^4$ the sum is larger when subnormal numbers are supported than when they are flushed to zero. This is because the terms are summed from smallest to largest, so that the smaller terms can contribute to the sum, even if they are subnormal [23], [24, sect. 4.2].

The next experiment emphasizes how quickly bfloat16 and fp16 can lose accuracy and the importance, again, of subnormal numbers. We solve the ordinary differential equation $y' = -y =: f(x, y)$ over $[0, 1]$, with $y(0) = 0.01$, using Euler's method, $y_{k+1} = y_k + hf(x_k, y_k)$, $y_0 = y(0)$, where $h = 1/n$, with several n between 10 and 10^5 . The absolute (global) errors $|y(1) - \hat{y}_n|$ are plotted in Figure 6.1. Since the global error in Euler's method is of order h , we expect a linear decrease in the error as n increases until rounding error starts to dominate, at which point the error will increase linearly, and the theory suggests that the optimal h is of order $u^{1/2}$, corresponding to an optimal n of order $u^{-1/2}$ [31, pp. 374–376]. This is roughly what we see in Figure 6.1: note that $u^{-1/2} = 16$ for bfloat16. However, perhaps surprising is that when subnormal numbers are flushed to zero in fp16 the errors start to grow much sooner than when subnormals are supported. In fact, once $n \gtrsim 100$ the $hf(x_k, y_k)$ term underflows and the computed y_k are constant when subnormals are not supported.

Our final experiment compares the different rounding modes in the solution of linear systems. We generate 100 linear systems $Ax = b$, with $A \in \mathbb{R}^{100 \times 100}$ and b having random fp16 elements from the normal (0,1) distribution. Each system is solved by LU factorization with partial pivoting and substitution using `lutx_chop`, with fp16 simulation and each of the supported rounding modes. Table 6.6 shows the mean, maximum, and minimum backward errors $\|b - A\hat{x}\|_1 / (\|A\|_1 \|x\|_1 + \|b\|_1)$ for the computed solutions $\hat{x}$. As might be expected, round to nearest produces the smallest backward errors.

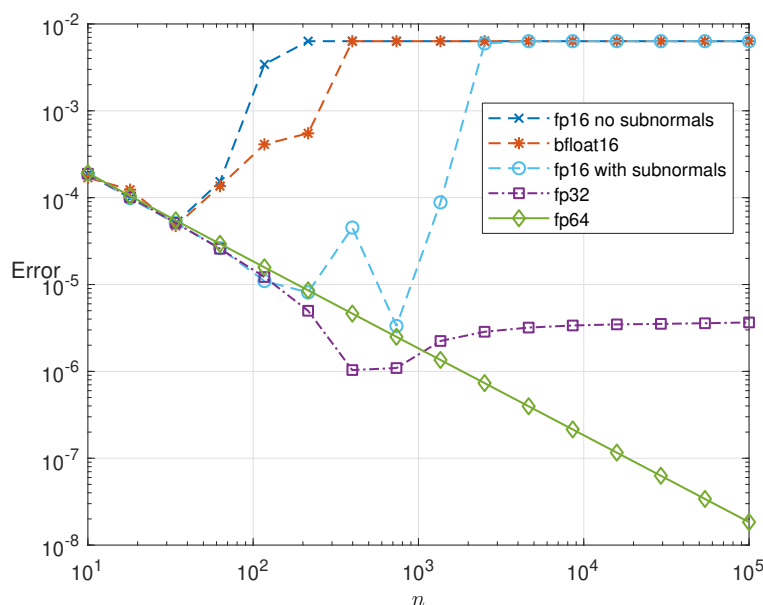


FIG. 6.1. Absolute errors in Euler's method for $y' = -y$, $y(0) = 0.01$, over $[0, 1]$, in different simulated arithmetics, with stepsize $h = 1/n$.

TABLE 6.6

Backward errors for 100 random linear systems $Ax = b$ with the elements of $A \in \mathbb{R}^{100 \times 100}$ and b from the normal $(0, 1)$ distribution. The solution is computed in fp16 arithmetic using LU factorization with partial pivoting and substitution. The stochastic rounding modes are defined in Table 5.1.

	Round to nearest	Round towards $+\infty$	Round towards $-\infty$	Round towards zero	Stochastic rounding 1	Stochastic rounding 2
Mean	5.24e-04	3.47e-03	3.50e-03	3.45e-03	6.86e-04	9.09e-04
Min	3.52e-04	1.87e-03	2.05e-03	1.94e-03	4.75e-04	6.55e-04
Max	7.74e-04	6.92e-03	5.72e-03	6.51e-03	1.44e-03	6.51e-03

7. Conclusions. We have analyzed the use of higher precision arithmetic with rounding to simulate lower precision arithmetic, showing that it is a valid way to simulate bfloat16 or fp16 using fp32 or fp64 arithmetic. It is essential to keep in mind the limitations pointed out in section 3, notably that results of greater accuracy than expected can be obtained if one carries out sequences of operations at higher precision before rounding back to the target format.

Our MATLAB function `chop` rounds to a specified target format, including fp16 or bfloat16 or a user-specified format, respecting precision and range, and offers six different rounding modes and optional support for subnormal numbers. It incurs a performance penalty of about one order of magnitude when applied to a pure MATLAB program file for LU factorization, compared with execution in double precision, which compares favorably with the overhead of Moler's fp16/vfp16 MATLAB class.

Software simulation provides a useful tool for understanding the effects of lowering the precision in numerical computations, as well as the benefits of representing subnormal numbers instead of flushing them to zero, as the examples in section 6 demonstrate. It also provides a convenient way to explore floating-point formats not

supported in hardware as well as stochastic rounding, which is becoming popular in machine learning.

Appendix A. The MATLAB function chop. The function `chop` uses the function `roundit`. Both functions are available from <https://github.com/higham/chop>. Test codes `test_chop` and `test_roundit` thoroughly test these functions. There are two main usages of `chop`. First, one can pass `options` with every call:

```
options.precision = 's'; options.round = 5; options.subnormal = 1;
...
A(i,j) = chop(A(i,j) - chop(A(i,k) * A(k,j),options),options);
```

Here, `options.precision = 's'` specifies that the precision is single, `options.round = 5` specifies stochastic rounding, mode 1 (see Table 5.1), and `options.subnormal = 1` specifies that subnormal numbers are not flushed to zero. For full details of the options see the link given at the start of the appendix. The above usage is rather tedious and produces cluttered code. Instead we can set up the arithmetic parameters on a call of the form `chop([],options)` and exploit the fact that subsequent calls with just one input argument will reuse the previously specified `options`:

```
options.precision = 's'; options.round = 5; options.subnormal = 1;
chop([],options)
...
A(i,j) = chop(A(i,j) - chop(A(i,k)*A(k,j)));
```

The current value of `options` is stored inside the function (in a persistent variable, whose value is retained during the session until the function is cleared with `clear chop`) and can be obtained with

```
[~,options] = chop
```

Acknowledgments. We thank the referees for their helpful comments on the manuscript.

REFERENCES

- [1] *ARM Architecture Reference Manual. ARMv8, for ARMv8-A Architecture Profile*, ARM Limited, Cambridge, UK, 2018, <https://developer.arm.com/docs/ddi0487/latest>, version dated 31 October 2018, original release dated 30 April 2013.
- [2] E. CARSON AND N. J. HIGHAM, *A new analysis of iterative refinement and its application to accurate solution of ill-conditioned sparse linear systems*, SIAM J. Sci. Comput., 39 (2017), pp. A2834–A2856, <https://doi.org/10.1137/17M1122918>.
- [3] E. CARSON AND N. J. HIGHAM, *Accelerating the solution of linear systems by iterative refinement in three precisions*, SIAM J. Sci. Comput., 40 (2018), pp. A817–A847, <https://doi.org/10.1137/17M1140819>.
- [4] M. CHANTRY, T. THORNES, T. PALMER, AND P. DÜBEN, *Scale-selective precision for weather and climate forecasting*, Monthly Weather Review, 147 (2019), pp. 645–655, <https://doi.org/10.1175/mwr-d-18-0308.1>.
- [5] A. J. COX AND N. J. HIGHAM, *Accuracy and stability of the null space method for solving the equality constrained least squares problem*, BIT, 39 (1999), pp. 34–50, <https://doi.org/10.1023/A:1022365107361>.
- [6] A. DAWSON AND P. D. DÜBEN, *rpe v5: An emulator for reduced floating-point precision in large numerical simulations*, Geoscientific Model Development, 10 (2017), pp. 2221–2230, <https://doi.org/10.5194/gmd-10-2221-2017>.

- [7] A. DAWSON, P. D. DÜBEN, D. A. MACLEOD, AND T. N. PALMER, *Reliable low precision simulations in land surface models*, *Climate Dynamics*, 51 (2018), pp. 2657–2666, <https://doi.org/10.1007/s00382-017-4034-x>.
- [8] J. DONGARRA, M. GATES, A. HAIDAR, J. KURZAK, P. LUSZCZEK, S. TOMOV, AND I. YAMAZAKI, *Accelerating numerical dense linear algebra calculations with GPUs*, in *Numerical Computations with GPUs*, V. Kindratenko, ed., Springer, Cham, 2014, pp. 3–28, https://doi.org/10.1007/978-3-319-06548-9_1.
- [9] S. DUTTA, Z. BAI, T. M. LOW, AND P. GROVER, *CodeNet: Training Large Scale Neural Networks in Presence of Soft-Errors*, preprint, <https://arxiv.org/pdf/1903.01042.pdf>, 2019.
- [10] M. FELDMAN, *Fujitsu Reveals Details of Processor That Will Power Post-K Supercomputer*, posted August 22, 2018; accessed November 22, 2018, <https://www.top500.org/news/fujitsu-reveals-details-of-processor-that-will-power-post-k-supercomputer>.
- [11] M. FELDMAN, *IBM Takes Aim at Reduced Precision for New Generation of AI Chips*, posted December 5, 2018; accessed January 8, 2019, <https://www.top500.org/news/ibm-takes-aim-at-reduced-precision-for-new-generation-of-ai-chips/>.
- [12] M. FELDMAN, *Intel Lays out Roadmap for Next Three Xeon Products*, posted August 9, 2018; accessed June 5, 2019, <https://www.top500.org/news/intel-lays-out-roadmap-for-next-three-xeon-products/>.
- [13] M. FELDMAN, *Record-Breaking Exascale Application Selected as Gordon Bell Finalist*, posted September 19, 2018; accessed January 8, 2019, <https://www.top500.org/news/record-breaking-exascale-application-selected-as-gordon-bell-finalist/>.
- [14] S. A. FIGUEROA, *When is double rounding innocuous?*, *SIGNUM News*, 30 (1995), pp. 21–26, <https://doi.org/10.1145/221332.221334>.
- [15] L. FOUSSE, G. HANROT, V. LEFÈVRE, P. PÉLISSIER, AND P. ZIMMERMANN, *MPFR: A multiple-precision binary floating-point library with correct rounding*, *ACM Trans. Math. Software*, 33 (2007), 13:1–13:15, <https://doi.org/10.1145/1236463.1236468>.
- [16] S. GUPTA, A. AGRAWAL, K. GOPALAKRISHNAN, AND P. NARAYANAN, *Deep learning with limited numerical precision*, in *Proceedings of the 32nd International Conference on Machine Learning*, *Proc. Mach. Learn. Res.* 37, 2015, pp. 1737–1746, <http://www.jmlr.org/proceedings/papers/v37/gupta15.html>.
- [17] A. HAIDAR, A. ABDELFAH, M. ZOUNON, P. WU, S. PRANESH, S. TOMOV, AND J. DONGARRA, *The design of fast and energy-efficient linear solvers: On the potential of half-precision arithmetic and iterative refinement techniques*, in *Computational Science—ICCS 2018*, Y. Shi et al., eds., Springer, Cham, 2018, pp. 586–600, https://doi.org/10.1007/978-3-319-93698-7_45.
- [18] A. HAIDAR, S. TOMOV, J. DONGARRA, AND N. J. HIGHAM, *Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers*, in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis—SC '18 (Dallas, TX)*, IEEE Press, Piscataway, NJ, 2018, 47:1–47:11, <https://doi.org/10.1109/SC.2018.00050>.
- [19] A. HAIDAR, P. WU, S. TOMOV, AND J. DONGARRA, *Investigating half precision arithmetic to accelerate dense linear system solvers*, in *Proceedings of the 8th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems—Scala '17 (Denver, CO)*, ACM, New York, 2017, 10, <https://doi.org/10.1145/3148226.3148237>.
- [20] S. HATFIELD, P. DÜBEN, M. CHANTRY, K. KONDO, T. MIYOSHI, AND T. PALMER, *Choosing the optimal numerical precision for data assimilation in the presence of model error*, *J. Adv. Model. Earth Syst.*, 10 (2018), pp. 2177–2191, <https://doi.org/10.1029/2018MS001341>.
- [21] N. J. HIGHAM, *The Matrix Computation Toolbox*, <http://www.maths.manchester.ac.uk/~higham/mctoolbox>.
- [22] N. J. HIGHAM, *The accuracy of solutions to triangular systems*, *SIAM J. Numer. Anal.*, 26 (1989), pp. 1252–1265, <https://doi.org/10.1137/0726070>.
- [23] N. J. HIGHAM, *The accuracy of floating point summation*, *SIAM J. Sci. Comput.*, 14 (1993), pp. 783–799, <https://doi.org/10.1137/0914050>.
- [24] N. J. HIGHAM, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, Philadelphia, 2002, <https://doi.org/10.1137/1.9780898718027>.
- [25] N. J. HIGHAM AND T. MARY, *A new preconditioner that exploits low-rank approximations to factorization error*, *SIAM J. Sci. Comput.*, 41 (2019), pp. A59–A82, <https://doi.org/10.1137/18M1182802>.
- [26] N. J. HIGHAM, S. PRANESH, AND M. ZOUNON, *Squeezing a matrix into half precision, with an application to solving linear systems*, *SIAM J. Sci. Comput.*, 41 (2019), pp. A2536–A2551, <https://doi.org/10.1137/18M1229511>.

- [27] S.-I. O'UCHI, H. FUKETA, T. IKEGAMI, W. NOGAMI, T. MATSUKAWA, T. KUDOH, AND R. TAKANO, *Image-classifier deep convolutional neural network training by 9-bit dedicated hardware to realize validation accuracy and energy efficiency superior to the half precision floating point format*, in Proceedings of the 2018 IEEE International Symposium on Circuits and Systems (ISCAS), Florence, Italy, 2018, pp. 1–5, <https://doi.org/10.1109/ISCAS.2018.8350953>.
- [28] *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985, IEEE Computer Society, New York, 1985.
- [29] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2008 (revision of IEEE Std 754-1985), IEEE Computer Society, New York, 2008, <https://doi.org/10.1109/IEEESTD.2008.4610935>.
- [30] INTEL CORPORATION, *BFLOAT16—Hardware Numerics Definition*, white paper, document number 338302-001US, 2018, <https://software.intel.com/en-us/download/bfloat16-hardware-numerics-definition>.
- [31] E. ISAACSON AND H. B. KELLER, *Analysis of Numerical Methods*, Wiley, New York, 1966; reprinted by Dover, New York, 1994.
- [32] *The Julia Programming Language*, <http://julialang.org>.
- [33] JULIA COMPUTING, *Bfloat16s*, <https://github.com/JuliaComputing/BFloat16s.jl>.
- [34] V. LEFÈVRE AND P. ZIMMERMANN, *Optimized binary64 and binary128 arithmetic with GNU MPFR*, in Proceedings of the 2017 IEEE 24th Symposium on Computer Arithmetic (ARITH), London, 2017, pp. 18–26, <https://doi.org/10.1109/ARITH.2017.28>.
- [35] D. MALONE, *To what does the harmonic series converge?*, Irish Math. Soc. Bulletin, 71 (2013), pp. 59–66, <https://www.maths.tcd.ie/pub/ims/bull71/recipe.pdf>.
- [36] C. B. MOLER, *Cleve's Laboratory*, <http://mathworks.com/matlabcentral/fileexchange/59085-cleve-laboratory>.
- [37] C. B. MOLER, *MATLAB Users' Guide*, Technical report CS81-1 (revised), Department of Computer Science, University of New Mexico, Albuquerque, NM, 1982.
- [38] C. B. MOLER, *"Half Precision" 16-Bit Floating Point Arithmetic*, posted May 8, 2017, <http://blogs.mathworks.com/cleve/2017/05/08/half-precision-16-bit-floating-point-arithmetic/>.
- [39] C. B. MOLER, *The Historic MATLAB Users' Guide*, posted February 5, 2018, <https://blogs.mathworks.com/cleve/2018/02/05/the-historic-matlab-users-guide/>.
- [40] C. B. MOLER, *Variable Format Half Precision Floating Point Arithmetic*, posted January 16, 2019, <https://blogs.mathworks.com/cleve/2019/01/16/variable-format-half-precision-floating-point-arithmetic/>.
- [41] J.-M. MULLER, *Elementary Functions: Algorithms and Implementation*, 3rd ed., Birkhäuser, Boston, MA, 2016, <https://doi.org/10.1007/978-1-4899-7983-4>.
- [42] *Multiprecision Computing Toolbox for MATLAB*, Advanpix, Tokyo, <http://www.advanpix.com>.
- [43] K. PALEM AND A. LINGAMNENI, *Ten years of building broken chips: The physics and engineering of inexact computing*, ACM Trans. Embed. Comput. Syst., 12 (2013), pp. 87:1–87:23, <https://doi.org/10.1145/2465787.2465789>.
- [44] T. N. PALMER, *More reliable forecasts with less precise computations: A fast-track route to cloud-resolved weather and climate simulators?*, Phil. Trans. R. Soc. A, 372 (2014), 20130391, <https://doi.org/10.1098/rsta.2013.0391>.
- [45] N. RAO, *Beyond the CPU or GPU: Why Enterprise-Scale Artificial Intelligence Requires a More Holistic Approach*, posted May 23, 2018; accessed November 5, 2018, <https://newsroom.intel.com/editorials/artificial-intelligence-requires-holistic-approach>.
- [46] C. RAU, *Half 1.12 IEEE 754-Based Half-Precision Floating Point Library*, <http://half.sourceforge.net/index.html>, Mar. 2017.
- [47] P. ROUX, *Innocuous double rounding of basic arithmetic operations*, J. Formalized Reasoning, 7 (2014), pp. 131–142, <https://doi.org/10.6092/issn.1972-5787/4359>.
- [48] S. M. RUMP, *INTLAB—Interval Laboratory*, in Developments in Reliable Computing, T. Csendes, ed., Kluwer Academic Publishers, Dordrecht, The Netherlands, 1999, pp. 77–104.
- [49] S. M. RUMP, *Verification methods: Rigorous results using floating-point arithmetic*, Acta Numer., 19 (2010), pp. 287–449, <https://doi.org/10.1017/S096249291000005X>.
- [50] S. M. RUMP, *IEEE754 precision-k base- β arithmetic inherited by precision-m base- β arithmetic for $k < m$* , ACM Trans. Math. Software, 43 (2017), 20:1–20:15, <https://doi.org/10.1145/2785965>.
- [51] V. SHAH, *Comment on Nicholas J. Higham, "Half Precision Arithmetic: fp16 Versus bfloat16,"* posted December 19, 2018, <https://nickhigham.wordpress.com/2018/12/03/half-precision-arithmetic-fp16-versus-bfloat16/#comment-5466>.

- [52] A. SVYATKOVSKIY, J. KATES-HARBECK, AND W. TANG, *Training distributed deep recurrent neural networks with mixed precision on GPU clusters*, in MLHPC'17: Proceedings of the Machine Learning on HPC Environments, ACM Press, New York, 2017, 10:1–10:8, <https://doi.org/10.1145/3146347.3146358>.
- [53] *Symbolic Math Toolbox*, The MathWorks, Inc., Natick, MA, <http://www.mathworks.co.uk/products/symbolic/>.
- [54] G. TAGLIAVINI, S. MACH, D. ROSSI, A. MARONGIU, AND L. BENIN, *A transprecision floating-point platform for ultra-low power computing*, in Proceedings of the IEEE 2018 Design, Automation and Test in Europe Conference and Exhibition (DATE), 2018, pp. 1051–1056, <https://doi.org/10.23919/DATE.2018.8342167>.
- [55] T. THORNES, P. DÜBEN, AND T. PALMER, *On the use of scale-dependent precision in earth system modeling*, Quart. J. Roy. Meteorol. Soc., 143 (2017), part B, pp. 897–908, <https://doi.org/10.1002/qj.2974>.
- [56] S. TOMOV, J. DONGARRA, AND M. BABOULIN, *Towards dense linear algebra for hybrid GPU accelerated manycore systems*, Parallel Comput., 36 (2010), pp. 232–240, <https://doi.org/10.1016/j.parco.2009.12.005>.
- [57] S. TOMOV, R. NATH, H. LTAIEF, AND J. DONGARRA, *Dense linear algebra solvers for multicore with GPU accelerators*, in Proceedings of the 2010 IEEE International Symposium on Parallel and Distributed Processing, Workshops and Phd Forum (IPDPS'10), Atlanta, GA, IEEE Press, Piscataway, NJ, pp. 1–8, <https://doi.org/10.1109/IPDPSW.2010.5470941>.
- [58] N. WANG, J. CHOI, D. BRAND, C.-Y. CHEN, AND K. GOPALAKRISHNAN, *Training deep neural networks with 8-bit floating point numbers*, in Advances in Neural Information Processing Systems 31 (NIPS 2018), S. Bengio et al., eds., Curran Associates, Inc., 2018, pp. 7675–7684, <http://papers.nips.cc/paper/7994-training-deep-neural-networks-with-8-bit-floating-point-numbers.pdf>.